

编译器优化测试用例

本文档提供了为类C语言编译器设计的优化测试用例，旨在验证编译器在不同优化阶段的性能和正确性。

1. 循环优化 (Loop Optimization)

这些测试用例旨在验证编译器对循环结构的优化能力，包括：

- 简单循环 (simple_loop.c):** 基本的while循环，测试编译器能否进行基本的循环优化。
- 循环不变代码外提 (invariant_code_motion.c):** 循环体内存在不依赖循环变量的表达式，测试编译器能否将其移到循环外部。
- 强度削减 (strength_reduction.c):** 循环体内存在乘法运算，测试编译器能否将其替换为加法运算。
- 循环展开 (loop_unrolling.c):** 测试编译器能否通过复制循环体来减少循环开销。
- 循环融合 (loop_fusion.c):** 多个相邻的独立循环，测试编译器能否将其合并为一个循环。

2. 常量传播 (Constant Propagation)

这些测试用例旨在验证编译器能否在编译时识别并替换程序中的常量值，从而简化表达式：

- 基本常量传播 (basic.c):** 变量被赋予常量值，然后该变量用于其他表达式。
- 函数调用中的常量传播 (function_call.c):** 常量作为函数参数传递。
- 条件语句中的常量传播 (conditional.c):** 条件表达式中包含常量。
- 循环中的常量传播 (loop.c):** 循环体内使用常量。
- 多重赋值 (multiple_assignments.c):** 变量通过多个赋值操作最终获得常量值。

3. 常量折叠 (Constant Folding)

这些测试用例旨在验证编译器能否在编译时计算出常量表达式的值：

- **算术运算 (arithmetic.c):** 包含加、减、乘、除、取模等算术运算的常量表达式。
- **逻辑运算 (logical.c):** 包含逻辑与、逻辑或、逻辑非等逻辑运算的常量表达式。
- **比较运算 (comparison.c):** 包含大于、小于、等于、不等于等比较运算的常量表达式。
- **混合运算 (mixed.c):** 包含算术和逻辑运算的混合常量表达式。
- **嵌套常量 (nested.c):** 包含多层嵌套的常量表达式。

4. 不可达代码消除 (Unreachable Code Elimination)

这些测试用例旨在验证编译器能否识别并移除永远不会被执行到的代码：

- **return语句后 (after_return.c):** return语句后面的代码。
- **永假条件 (false_condition.c):** if语句的条件永远为假，导致其内部代码不可达。
- **break后 (after_break.c):** break语句后面的循环体代码。
- **continue后 (after_continue.c):** continue语句后面的循环体代码。
- **未调用的函数 (dead_function.c):** 定义了但从未被调用的函数。

5. 死代码消除 (Dead Code Elimination)

这些测试用例旨在验证编译器能否识别并移除其结果永不被使用的代码：

- **未使用的变量 (unused_variable.c):** 变量被赋值但从未被读取。
- **未使用的赋值 (unused_assignment.c):** 对变量的赋值操作，但该赋值的结果在后续代码中未被使用或被覆盖。
- **不可达的赋值 (unreachable_assignment.c):** 位于不可达代码块中的赋值操作。
- **未使用的函数结果 (unused_function_result.c):** 函数被调用，但其返回值未被使用。
- **复杂死代码 (complex_dead_code.c):** 包含条件分支和多重赋值的复杂死代码场景。

6. 窥孔优化 (Peephole Optimization)

这些测试用例旨在验证编译器能否在局部范围内对指令序列进行优化，例如：

- **冗余加载/存储 (redundant_load_store.c):** 变量被赋值后立即被另一个变量读取，测试能否消除冗余的内存操作。
- **冗余跳转 (redundant_jump.c):** 连续的跳转指令或不必要的跳转。
- **代数简化 (algebraic_simplification.c):** 识别并简化如 $x + 0$ 、 $x * 1$ 等代数恒等式。
- **指令合并 (combine_instructions.c):** 将多条简单指令合并为一条更高效的指令。
- **局部常量传播 (constant_propagation_local.c):** 在小范围内进行常量传播。

7. 高强度寄存器使用 (High Strength Register Usage)

这些测试用例旨在验证编译器能否有效地利用寄存器来存储和操作数据，减少内存访问，从而提高性能：

- **复杂算术 (complex_arithmetic.c):** 包含多个变量和复杂算术运算的表达式，测试寄存器分配。
- **函数参数 (function_params.c):** 包含大量参数的函数调用，测试参数传递和寄存器使用。
- **循环变量 (loop_variables.c):** 嵌套循环中的多个循环变量，测试寄存器分配和重用。
- **多返回值 (multiple_returns.c):** 函数返回多个值（通过参数或结构体模拟），测试返回值处理。
- **链式操作 (chained_operations.c):** 对同一个变量进行连续操作，测试寄存器重用。

请使用这些测试用例来评估您的编译器在不同优化方面的性能。